

is returned directly to the user, eliminating the need for another LLM call. If no suitable match is found, the request proceeds through the complete retrieval-augmented generation (RAG) pipeline, where relevant documents are retrieved before the LLM generates a response. New response is then stored alongside its embedding, improving future cache performance.

Because responses are increasingly served from the cache rather than generated repeatedly, both inference latency and token consumption decrease over time.

Demonstrating semantic caching

A demonstration using a streaming service support assistant illustrates this behaviour.

In the first interaction, a user asks how to download a movie for offline viewing. The request passes through an RAG pipeline that retrieves relevant policy documents before the LLM generates a response. This process consumes 701 tokens and takes approximately 5.3 seconds.

A second user later asks how to watch a movie without an internet connection. Although the wording is completely different, both questions have the same meaning. Rather than invoking the LLM again, the application retrieves the response directly from the semantic cache.

As a result, no additional tokens are consumed, and response latency falls dramatically to around 700 milliseconds. Since the request is served directly from Redis instead of passing through the LLM, inference delays and network overheads are largely eliminated.

This highlights how semantic caching simultaneously reduces operating costs and improves the user experience by delivering faster responses without sacrificing accuracy.

Redis: Beyond traditional caching

Although Redis is widely recognised as

an in-memory caching platform, it also supports vector data structures, allowing it to function as a vector database for AI applications. This enables developers to combine semantic caching, vector search and real-time data processing within a single platform.

By storing vector embeddings alongside cached responses, Redis supports semantic similarity searches while maintaining the low-latency performance required for production AI applications.

Real-world applications of semantic caching

Semantic caching is already being used in production environments to reduce LLM costs while improving application performance.

One example is Asurion, a device insurance provider serving approximately 300 million customers. By combining Redis Vector Database, semantic caching and a semantic router, the company achieved a 70 per cent reduction in LLM costs. Frequently asked questions were served directly from the semantic cache, while the semantic router determined the most appropriate processing path for incoming requests.

Another example is Mangos AI, a voice AI healthcare application. The company also reported a 70 per cent cache hit ratio and a corresponding 70 per cent reduction in LLM costs by using LangCache, Redis' managed semantic caching service. LangCache manages embedding generation and cache operations automatically, allowing developers to integrate semantic caching through a REST API.

Introducing the semantic router

Many AI applications use an LLM to determine a user's intent before deciding which tool to invoke or which knowledge source to query. Although effective, this additional LLM call increases both latency and cost.

A semantic router provides an alternative approach by performing

intent classification using vector similarity instead of an LLM. Incoming queries are converted into embeddings and compared against predefined intent vectors, allowing the application to route requests without invoking a language model.

By relying on vector search rather than generative inference, semantic routing produces deterministic results while reducing response time and token consumption.

Design principle 1: Choose the right embedding model

The effectiveness of semantic caching depends heavily on the quality of the embedding model used to represent user queries (Figure 1).

Rather than selecting a model based on general recommendations, developers should evaluate multiple embedding models using their own datasets. Benchmarking against real application data provides a more accurate indication of retrieval quality than relying solely on published benchmark results.

An example is the LangCache Embed V2 model, which has been fine-tuned specifically for semantic caching. Unlike general-purpose embedding models, it is designed to distinguish between queries that genuinely share the same intent and those that are merely topically related.

For example, questions such as "What is the refund policy?" and "How do I get my money back?" should generate similar embeddings because they require the same answer. By contrast, questions such as "What is the capital of Australia?" and "What is the largest city in Australia?" discuss the same country but require different responses. A generic embedding model may place these questions close together because they share geographical context, whereas an embedding model optimised for semantic caching is more likely to distinguish between them correctly.